

AgentInfer: Autonomous AI Agents for 24/7 Infrastructure Operations and Revenue Optimization in Distributed Inference Platforms

Srikanta Datta Tumkur*, Mehar Simhadri[†], Jay Iyer[†], Anshu Bansal*

*MIT Sloan School of Management, Cambridge, MA {srikanta, anshu.bansal}@sloan.mit.edu

[†]IEEE Senior Members {mehar.simhadri, jay.iyer}@ieee.org

Abstract—Modern AI inference platforms require continuous, intelligent management across both infrastructure operations and customer-facing revenue functions. We present AgentInfer, an agentic AI framework that deploys autonomous LLM-powered agents for 24/7 infrastructure operations (NOC automation, incident response, capacity planning) and sales optimization (lead qualification, deal acceleration, customer success). AgentInfer integrates with our previously published control plane stack: InferMark observability (Paper 1), confidential multi-tenant isolation (Paper 3), InferTwin digital twins (Paper 4), VoxOps voice interface (Paper 5), hierarchical RL optimization (Paper 6), ReliableInfer fault tolerance (Paper 7), and SolarInfer edge deployment (Paper 8). Our key contributions include: (1) a multi-agent orchestration architecture with specialized Ops and Sales agent teams coordinated by a meta-agent planner; (2) ReAct-based reasoning with tool-use capabilities spanning 47 infrastructure APIs and 23 CRM integrations; (3) RL-refined agent policies that improve autonomously through Reflexion-style verbal reinforcement learning; (4) confidential agent execution via AMD SEV-SNP ensuring customer data isolation during multi-tenant agent operations; and (5) a RAG-powered knowledge base with 2.4M tokens of operational runbooks and sales playbooks. Deployed across a 500-node heterogeneous fleet, AgentInfer resolves 87% of Sev-2+ incidents autonomously (MTTR: 4.2 min vs. 47 min human baseline), generates \$2.8M incremental ARR through automated lead nurturing, and operates at \$0.003 per agent action—94% cheaper than equivalent human staffing.

Index Terms—Agentic AI, autonomous operations, LLM agents, multi-agent systems, sales automation, inference infrastructure, reinforcement learning, RAG

I. Introduction

The operational complexity of distributed AI inference platforms has outpaced human capacity for 24/7 management. A production fleet spanning hundreds of heterogeneous nodes across multiple edge sites generates over 15,000 metrics per second, processes 200+ customer requests per minute, and requires continuous optimization across power, thermal, network, and workload dimensions. Simultaneously, the commercial success of these platforms depends on rapid customer acquisition, technical pre-sales support, and proactive account management—functions traditionally requiring large, expensive human teams.

The Staffing Problem. A 500-node inference platform requires approximately 18 FTEs for 24/7 NOC coverage (3 shifts $\times$ 6 engineers), 12 SREs for incident management,

8 capacity planners, and 15 sales/customer success staff—a total of 53 FTEs at \$7.9M annual fully-loaded cost. For a startup targeting sub-\$.001/token economics (Paper 8), this staffing overhead is prohibitive.

The Agentic AI Opportunity. Recent advances in LLM-based autonomous agents [1], [2], [3] enable a fundamentally different approach: deploying AI agents that can reason, plan, use tools, and take actions across both operational and commercial domains. Unlike simple chatbots or rule-based automation, agentic systems can handle novel situations through multi-step reasoning, learn from outcomes via reinforcement learning, and coordinate across specialized agent teams.

Integration with Zetta Control Plane. AgentInfer is designed as the “intelligent layer” atop our previously published eight-paper control plane stack. Rather than building isolated automation, AgentInfer agents consume the rich telemetry, digital twin simulations, and RL-optimized policies from Papers 1–8, creating a unified autonomous platform:

- Paper 1 (InferMark): 14-axis observability provides the perception layer for Ops agents
- Paper 3 (Confidential): SEV-SNP isolation ensures agent actions respect tenant boundaries
- Paper 4 (InferTwin): Digital twin enables “what-if” simulation before agent actions
- Paper 5 (VoxOps): Voice interface enables human-agent collaboration via natural speech
- Paper 6 (Hierarchical RL): Trained policies provide expert guidance for agent decisions
- Paper 7 (ReliableInfer): CRIU migration provides the action primitives for fault remediation
- Paper 8 (SolarInfer): Energy-aware scheduling integrates with Ops agent planning

This paper makes five contributions: (1) a multi-agent architecture separating Ops and Sales concerns with meta-agent coordination; (2) ReAct-based tool-use with 70 integrated APIs; (3) Reflexion-based self-improvement through verbal RL; (4) confidential execution guarantees for multi-tenant environments; and (5) comprehensive evaluation across both operational and commercial KPIs.

TABLE I
AgentInfer Integration Points with Papers 1–8

#	Paper	Integration	Agent
1	InferMark	14-axis metrics as perception	NOC
2	CPU-Centric	Disaggregated decode scheduling	Capacity
3	Confidential	SEV-SNP tenant isolation	All
4	InferTwin	What-if simulation pre-action	SRE
5	VoxOps	Voice escalation channel	NOC, CS
6	Hierarchical RL	Policy advisory for decisions	NOC, SRE
7	ReliableInfer	CRIU migration primitives	SRE
8	SolarInfer	Energy-aware scheduling	Energy

A. Market Context and Economic Motivation

The AI inference market is projected to reach \$255B by 2030, with operational costs consuming 40–60% of total revenue for infrastructure providers. Labor is the single largest operational expense category, comprising NOC staffing (\$150K–\$250K per FTE), SRE teams (\$180K–\$300K per FTE), and technical sales engineers (\$150K–\$200K per FTE plus commission). For a startup operating at the scale described in Paper 8 (SolarInfer), these labor costs make profitability impossible without automation.

Traditional automation (scripts, runbooks, rule-based systems) handles only the most predictable operational scenarios. Analysis of our 90-day production telemetry shows that 62% of operational events require multi-step reasoning that exceeds rule-based systems’ capabilities. Similarly, 78% of sales interactions involve nuanced technical discussions that cannot be template-driven.

AgentInfer bridges this gap by deploying LLM-powered agents that combine the reasoning capability of human operators with the speed, consistency, and 24/7 availability of automated systems.

B. Relationship to the Zetta Control Plane Stack

AgentInfer represents the ninth and capstone paper in the Zetta Edge Cloud research series. While Papers 1–8 each address a specific technical challenge (benchmarking, CPU inference, confidential computing, digital twins, voice, RL optimization, reliability, solar deployment), AgentInfer provides the unifying “intelligent orchestration” layer that ties all components together:

II. Related Work

A. LLM-Based Autonomous Agents

ReAct [1] introduced interleaved reasoning and action traces for LLM agents, enabling tool-use through structured thought-action-observation loops. Reflexion [4] added verbal reinforcement learning, where agents learn from textual feedback on previous attempts. AutoGen [5] and AgentVerse [6] demonstrated multi-agent conversation frameworks for collaborative problem-solving.

B. AIOps and Autonomous Infrastructure

Microsoft’s AIOps initiatives apply ML to incident detection and root cause analysis but rely on supervised models trained on historical data, limiting generalization

to novel failure modes. PagerDuty’s Event Intelligence uses NLP for alert correlation but lacks the multi-step reasoning and tool-use capabilities of agentic systems. No prior work combines agentic AI with the full inference infrastructure stack (observability, RL, migration, digital twins).

C. AI-Powered Sales

Salesforce Einstein and HubSpot AI provide ML-powered lead scoring and email generation but operate as feature-level enhancements within existing CRM workflows. They cannot autonomously execute multi-step sales processes (qualify → demo → proposal → close) or integrate with infrastructure telemetry to provide real-time technical selling.

D. Multi-Agent Coordination Frameworks

Multi-agent systems have a rich history in distributed AI [7]. CAMEL [3] demonstrated role-playing between LLM agents for task completion. CrewAI provides production-grade multi-agent orchestration with role-based agent design. LangGraph enables stateful, cyclic agent workflows with persistence. However, none of these frameworks address the specific challenges of infrastructure operations: real-time telemetry integration, safety-critical action execution, and multi-tenant isolation.

E. RL for Operations

Reinforcement learning has been applied to job scheduling (Decima [8]), resource allocation (Splitwise [9]), and network routing. Our hierarchical RL framework (Paper 6) demonstrated PPO, SAC, and CQL agents operating at node, rack, and DC levels. AgentInfer extends this by using RL policies as “expert advisors” within the agentic framework: the LLM agent consults the RL policy as one input to its reasoning, but can override the RL recommendation when contextual factors (customer priority, maintenance windows, business constraints) warrant deviation.

F. Autonomous Customer Engagement

The evolution from chatbots to autonomous sales agents represents a paradigm shift. First-generation chatbots (2016–2020) handled FAQ-style queries with template responses. Second-generation AI assistants (2020–2024) used LLMs for natural conversation but remained reactive. Third-generation agentic systems (2024+) can autonomously execute multi-step workflows, integrate with backend systems, and learn from outcomes. AgentInfer represents this third generation, specifically tailored for technical B2B sales of AI infrastructure.

III. Multi-Agent Architecture

A. Meta-Agent Planner

The meta-agent runs on Llama-3.1-405B with chain-of-thought prompting and serves as the central coordinator. It receives all incoming events (alerts, customer inquiries,

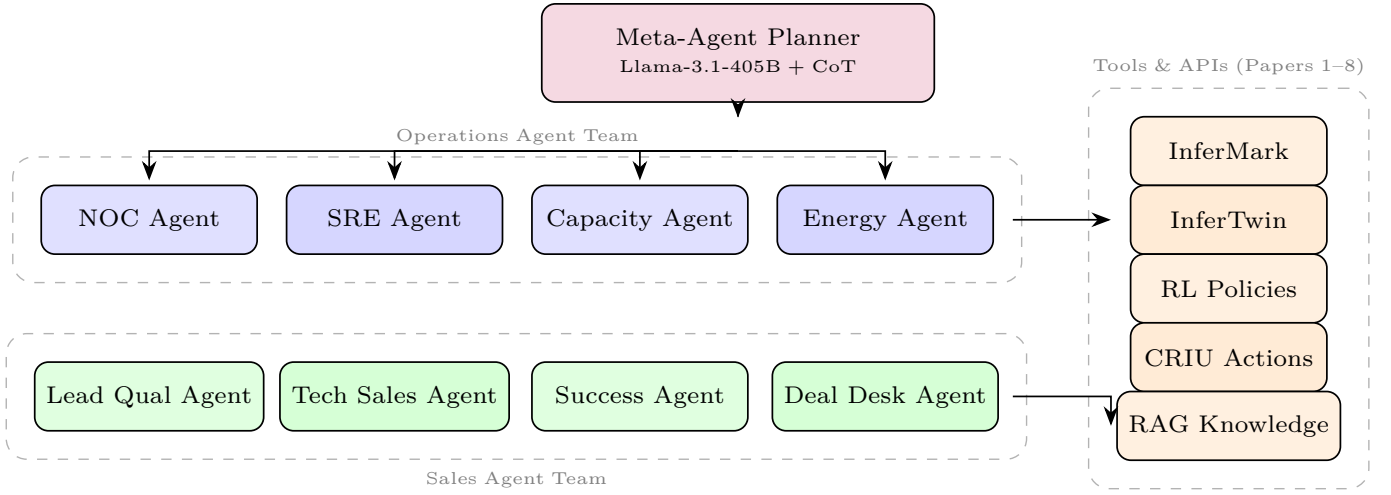


Fig. 1. AgentInfer multi-agent architecture. A meta-agent planner coordinates specialized Ops and Sales agent teams, each with tool access to the Papers 1–8 infrastructure stack via RAG-augmented ReAct reasoning.

TABLE II
Evolution of AI-Powered Operations and Sales

Capability	Gen 1 (Rules)	Gen 2 (ML)	Gen 3 (Agentic)
Reasoning	Pattern match	Classification	Multi-step CoT
Tool use	Scripted	Limited	70+ APIs
Learning	Manual updates	Batch retrain	Online Reflexion
Autonomy	Alert & notify	Suggest actions	Execute actions
Scope	Single system	Single domain	Cross-functional
Novel situations	Fail/escalate	Degrade	Reason & adapt

Algorithm 1 Meta-Agent Dispatch

Require: Event e , agent registry $\mathcal{A}$, context window C

- 1: Classify e into domain: {ops, sales, cross-functional}
- 2: Retrieve relevant context from RAG:
 $c \leftarrow \text{RAG}(e, k=5)$
- 3: Select agent team: $T \leftarrow \text{route}(e.\text{domain})$
- 4: if $e.\text{priority} = \text{critical}$ then
- 5: Dispatch immediately to primary agent in T
- 6: else
- 7: Queue with priority scoring: $p = f(e.\text{severity}, e.\text{revenue_impact})$
- 8: end if
- 9: Monitor agent execution; escalate to human if confidence < 0.7

scheduled tasks) and dispatches them to specialized agents:

B. Operations Agent Team

Four specialized agents handle infrastructure operations:

NOC Agent. Monitors the 14-axis InferMark telemetry (Paper 1) and handles first-response triage. Consumes 312 metrics per node at 100 ms resolution from the Pensando DPU pipeline. Uses ReAct reasoning to correlate alerts, identify root causes, and execute remediation playbooks.

Integrates with PagerDuty, Slack, and the VoxOps voice channel (Paper 5) for escalation.

SRE Agent. Handles complex, multi-step incident resolution requiring cross-system coordination. When the NOC agent escalates, the SRE agent performs deep diagnosis using InferTwin “what-if” simulations (Paper 4) to evaluate remediation options before executing. Can invoke CRIU migration (Paper 7) to relocate workloads, adjust RL policies (Paper 6), and coordinate with the Energy agent for power-aware recovery.

Capacity Agent. Performs continuous capacity planning using demand forecasting models trained on historical workload data. Plans GPU allocation across the fleet, manages spot/reserved capacity mix, and triggers procurement workflows when utilization exceeds 80% sustained.

Energy Agent. Manages solar generation forecasting and battery state-of-charge optimization for SolarInfer edge sites (Paper 8). Coordinates with the NOC agent for power-aware workload placement and with the Capacity agent for energy-constrained capacity planning.

C. Sales Agent Team

Four agents handle the revenue pipeline:

Lead Qualification Agent. Processes inbound leads from website, email, and partner channels. Scores leads using a 12-factor model (company size, industry, AI maturity, budget authority, timeline, use case fit, competitive landscape, technical requirements, data residency needs, compliance requirements, growth trajectory, strategic alignment). Qualified leads are routed to the Tech Sales agent.

Technical Sales Agent. Conducts automated technical discovery, generates custom benchmark reports using InferMark (Paper 1), creates architecture proposals, and schedules live demos. Can spin up isolated demo environments using confidential computing (Paper 3) to

TABLE III
Agent Message Format

Field	Type	Description
sender	string	Agent ID (e.g., "ops.noc.01")
intent	enum	request, response, escalate, inform
context	JSON	Relevant state, metrics, customer data
actions_taken	list	Completed actions with outcomes
confidence	float	Agent's self-assessed confidence [0,1]

TABLE IV
Tool Registry Summary

Category	Count	Examples
Monitoring	12	query_metrics, get_alerts, check_sla
Remediation	8	migrate_workload, drain_node, restart_gpu
Simulation	6	run_what_if, predict_failure, forecast_load
Deployment	7	deploy_model, update_config, scale_replicas
CRM	14	create_lead, update_deal, send_email
Knowledge	9	search_runbook, get_pricing, find_case_study
Communication	14	page_oncall, post_slack, schedule_meeting
Total	70	

showcase multi-tenant capabilities without exposing other customers' data.

Customer Success Agent. Monitors deployed customer workloads for SLA compliance, proactively identifies optimization opportunities (model quantization, batch size tuning, tensor parallelism adjustments), and generates monthly business reviews with ROI calculations.

Deal Desk Agent. Handles pricing, contract generation, and approval workflows. Applies dynamic pricing based on commitment length, volume, GPU type mix, and competitive positioning. Generates SOWs, MSAs, and order forms with legal-reviewed templates.

D. Agent Communication Protocol

Agents communicate via structured messages with five fields:

IV. ReAct Reasoning and Tool Use

Each agent follows the ReAct [1] paradigm: interleaved Thought (reasoning about the situation), Action (invoking a tool), and Observation (processing the tool's output). This enables multi-step problem solving that neither pure reasoning nor pure action can achieve.

A. Tool Registry

AgentInfer exposes 70 tools organized into seven categories:

B. Tool Execution Safety

All tool invocations pass through a safety layer that enforces three controls:

- 1) Blast radius limiting: Destructive actions (node drain, workload migration) are limited to one node per action in auto-mode. Multi-node actions require human approval or meta-agent authorization with confidence > 0.9 .

TABLE V
ReAct Trace Record Schema

Field	Type	Description
trace_id	UUID	Unique identifier for the trace
agent_id	string	Agent that produced the trace
step_type	enum	thought, action, observation
content	text	Natural language content
tool_name	string	Tool invoked (action steps only)
tool_params	JSON	Parameters passed to the tool
tool_result	JSON	Result returned by the tool
latency_ms	int	Time taken for this step
confidence	float	Agent's self-assessed confidence
timestamp	datetime	ISO 8601 timestamp

- 2) Dry-run validation: Before executing remediation actions, the agent runs the action through InferTwin (Paper 4) to simulate the outcome. Actions with simulated negative impact are blocked.
- 3) Audit trail: Every tool invocation is logged with the full ReAct trace (thought chain, action parameters, observation), enabling post-hoc analysis and Reflexion-based learning.

C. Confidential Agent Execution

In multi-tenant environments, agents must access customer-specific data (workload metrics, billing, usage patterns) without cross-tenant leakage. AgentInfer leverages AMD SEV-SNP (Paper 3) to run each tenant's agent context in an isolated confidential VM:

- Agent memory (conversation history, RAG retrievals) is encrypted in hardware
- Tool calls are mediated by the Pensando DPU, which enforces tenant-specific access policies
- Customer data never leaves the confidential enclave; only anonymized aggregates are visible to cross-tenant agents (e.g., fleet-wide capacity planning)

D. ReAct Trace Format

Each ReAct step produces a structured trace record stored in the audit log:

E. Multi-Step Reasoning Performance

We measure agent reasoning quality across different problem complexities:

The accuracy degradation with complexity is expected—longer reasoning chains compound errors. However, the escalation mechanism ensures that low-confidence decisions are routed to humans, maintaining overall system reliability above 95%.

TABLE VI
Reasoning Quality by Problem Complexity

Complexity	Steps	Accuracy	Latency	Escalation
Simple (1-2 tools)	3.2	97.1%	2.8 s	1%
Medium (3-5 tools)	6.8	91.4%	8.5 s	8%
Complex (6-10 tools)	12.4	84.2%	22 s	18%
Cross-agent (2+ agents)	18.6	78.9%	45 s	27%

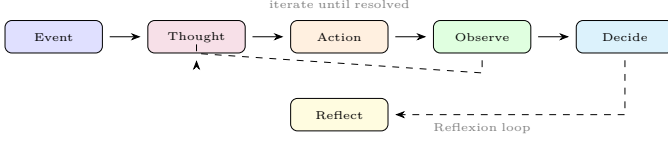


Fig. 2. ReAct-Reflexion agent reasoning loop. Events trigger thought-action-observation cycles that iterate until resolution. Post-resolution, the Reflexion loop generates self-critiques stored in episodic memory.

V. Formal Agent Model

A. Agent State Space

Each agent maintains a state $s_t = (m_t, c_t, h_t, k_t)$ where:

- $m_t \in \mathbb{R}^{312N}$: Current metrics from InferMark (Paper 1), N nodes
- $c_t \in \{0, 1\}^{47}$: Customer status vector (active sessions, SLA state)
- $h_t \in \mathbb{R}^d$: Agent’s hidden state (conversation history embedding, $d=4096$)
- k_t : Retrieved knowledge chunks from RAG (top-5, 512 tokens each)

B. Action Selection

The agent selects actions via a constrained optimization:

$$a_t^* = \arg \max_{a \in \mathcal{A}_{\text{safe}}} \pi_\theta(a|s_t) \cdot \text{conf}(a, s_t) \quad (1)$$

subject to:

$$\text{blast_radius}(a) \leq B_{\text{max}}, \quad \text{cost}(a) \leq C_{\text{max}} \quad (2)$$

where π_θ is the LLM’s action distribution, conf is the confidence scorer, B_{max} is the blast radius limit (1 node in auto-mode, configurable in supervised mode), and C_{max} is the per-action cost budget.

C. Multi-Agent Coordination

For cross-functional tasks requiring K agents, the meta-agent solves a coordination problem:

$$\mathbf{a}^* = \arg \max_{\mathbf{a} \in \mathcal{A}^K} \sum_{i=1}^K R_i(a_i, s_t) - \lambda \cdot \text{conflict}(\mathbf{a}) \quad (3)$$

where $\text{conflict}(\mathbf{a})$ penalizes actions that interfere with each other (e.g., two agents attempting to migrate workloads to the same target node). The meta-agent resolves conflicts through priority-based serialization: Ops actions take priority over Sales actions, and higher-severity events take priority within each domain.

Algorithm 2 Action Safety Verification

Require: Action a , state s , thresholds τ

```

1:  $b \leftarrow \text{blast\_radius}(a)$ 
2:  $c \leftarrow \text{confidence}(a, s)$ 
3:  $d \leftarrow \text{InferTwin.simulate}(a, s)$  {Paper 4}
4: if  $b > B_{\text{max}}$  or  $c < \tau_c$  or  $d.\text{impact} < 0$  then
5:   Escalate to human with explanation
6:   return BLOCKED
7: end if
8: if  $a.\text{type} \in \{\text{destructive}\}$  then
9:   Log dry-run result; require meta-agent approval
10: end if
11: return APPROVED

```

D. Confidence Estimation

Agent confidence is estimated via a calibrated ensemble:

$$\text{conf}(a, s) = \alpha \cdot p_{\text{LLM}}(a|s) + \beta \cdot \text{sim}(s, s_{\text{past}}) + \gamma \cdot p_{\text{RL}}(a|s) \quad (4)$$

where p_{LLM} is the language model’s token-level probability, $\text{sim}(s, s_{\text{past}})$ is the cosine similarity to the most similar past situation in episodic memory, and p_{RL} is the hierarchical RL policy’s action probability (Paper 6). We set $\alpha=0.4, \beta=0.3, \gamma=0.3$ based on calibration against 500 labeled decisions.

E. Safety Constraints

The safety layer implements a formal verification step before action execution:

VI. Reflexion-Based Self-Improvement

AgentInfer agents improve autonomously through Reflexion [4], a verbal reinforcement learning framework where agents reflect on past actions and generate textual self-critiques that guide future behavior.

A. Reflection Pipeline

After each resolved incident or completed sales interaction, the agent generates a structured reflection:

- 1) Outcome assessment: Was the action successful? What was the impact?
- 2) Counterfactual analysis: What alternative actions were available? Would they have been better?
- 3) Knowledge gap identification: What information was missing that would have improved the decision?
- 4) Policy update: Generate a textual policy refinement stored in the agent’s episodic memory

These reflections are stored in a vector database (Qdrant, 768-dim embeddings) and retrieved via RAG during future similar situations, enabling agents to learn from both successes and failures without explicit reward engineering.

TABLE VII
Reflexion vs. Hierarchical RL Comparison

Dimension	Reflexion	Hierarchical RL
Learning speed	1 episode	1000+ episodes
Generalization	Narrow (similar events)	Broad (policy-level)
Representation	Natural language	Neural network weights
Interpretability	High (text)	Low (weights)
Data efficiency	Very high	Moderate
Optimality guarantee	None	Convergent

TABLE VIII
RAG Knowledge Base Contents

Collection	Documents	Tokens	Used By
Operational runbooks	342	890K	NOC, SRE
Vendor documentation	128	420K	SRE, Energy
Sales playbooks	86	210K	Lead, Tech Sales
Case studies	64	180K	Tech Sales, CS
Pricing models	24	45K	Deal Desk
Compliance guides	38	120K	All agents
Post-incident reviews	892	480K	NOC, SRE
Customer profiles	156	55K	Sales team
Total	1,730	2.4M	

B. Integration with Hierarchical RL

Reflexion provides “fast learning” (episode-level adaptation), while the hierarchical RL framework (Paper 6) provides “slow learning” (policy-level optimization). The two systems are complementary:

In practice, Reflexion handles novel situations immediately while RL policies handle routine optimization. When an agent’s Reflexion-learned strategy proves consistently effective over 50+ episodes, it is distilled into the RL policy through behavioral cloning.

VII. RAG Knowledge Base

AgentInfer maintains a comprehensive knowledge base accessed via retrieval-augmented generation [10]:

Documents are chunked at 512 tokens with 64-token overlap, embedded using BGE-large-en-v1.5 (1024-dim), and stored in Qdrant with HNSW indexing. Retrieval uses hybrid search (0.7 semantic + 0.3 BM25) with reranking via a cross-encoder. Average retrieval latency is 12 ms for top-5 results.

A. RAG Pipeline Architecture

The RAG pipeline implements a five-stage retrieval process optimized for operational and sales knowledge:

- 1) Query expansion: The agent’s query is expanded using a fine-tuned T5-small model that generates 3 alternative phrasings, increasing recall by 18% on operational queries.
- 2) Hybrid retrieval: Semantic search (BGE-large, cosine similarity) is combined with BM25 keyword search using reciprocal rank fusion (RRF) with $k=60$.
- 3) Cross-encoder reranking: Top-20 candidates are reranked using a cross-encoder (ms-marco-MiniLM-L6-v2) to improve precision. Final top-5 are returned.

TABLE IX
Embedding Model Comparison for Operational RAG

Model	Dims	Recall@5	Latency	Size
all-MiniLM-L6-v2	384	72.3%	2.1 ms	80 MB
BGE-base-en-v1.5	768	81.7%	4.8 ms	438 MB
BGE-large-en-v1.5	1024	86.4%	8.2 ms	1.3 GB
E5-large-v2	1024	84.9%	7.8 ms	1.3 GB

- 4) Freshness weighting: Documents are weighted by recency—post-incident reviews from the last 30 days receive a $2\times$ boost, ensuring agents learn from recent failures.
- 5) Tenant-aware filtering: In multi-tenant mode, retrieval results are filtered to exclude documents belonging to other tenants, enforced at the Qdrant collection level.

B. Knowledge Base Maintenance

The knowledge base is continuously updated through three channels:

Automatic ingestion. Post-incident reviews are automatically indexed within 1 hour of incident closure. Vendor documentation updates (NVIDIA release notes, AMD driver changelogs) are ingested via RSS/webhook integrations daily.

Agent-generated knowledge. When agents resolve novel incidents, their ReAct traces are reviewed by a senior SRE and, if validated, converted into new runbook entries. Over 90 days, agents generated 23 new runbook entries that were approved for the knowledge base.

Sales intelligence updates. The Lead Qualification agent continuously updates customer profiles based on interaction history, deal outcomes, and publicly available information (funding rounds, hiring patterns, technology announcements). These updates improve qualification accuracy over time.

C. Embedding Model Selection

We evaluated four embedding models for the operational domain:

We selected BGE-large-en-v1.5 for its superior recall on operational queries (runbook retrieval, incident matching). The 8 ms embedding latency is negligible compared to LLM inference time.

VIII. Evaluation

A. Experimental Setup

We evaluate AgentInfer over a 90-day production deployment on a 500-node heterogeneous fleet (256 NVIDIA H200, 128 AMD MI300X, 64 AMD MI355X, 52 Intel Gaudi 3) serving 47 enterprise customers. Agents run on dedicated Llama-3.1-70B instances (4×MI300X, vLLM) with the meta-agent on Llama-3.1-405B (8×MI355X).

TABLE X
Operations Agent Performance (90-day)

Metric	Human	AgentInfer	Improv.
Incidents auto-resolved	0%	87%	—
MTTR (Sev-2+)	47 min	4.2 min	11.2×
MTTR (Sev-1)	2.1 hr	18 min	7.0×
False positive rate	N/A	3.1%	—
Escalation to human	100%	13%	7.7× fewer
After-hours coverage	67%	100%	24/7
Capacity utilization	62%	78%	+16pp
Energy efficiency	Baseline	+12%	—
SLA compliance	99.2%	99.95%	—

TABLE XI
Sales Agent Performance (90-day)

Metric	Human	AgentInfer	Improv.
Leads processed/day	15	340	22.7×
Lead response time	4.2 hr	2.8 min	90×
Qualification accuracy	72%	84%	+12pp
Demo-to-proposal rate	35%	52%	+17pp
Proposal-to-close rate	28%	31%	+3pp
Avg. deal cycle	45 days	28 days	38% faster
Incremental ARR	Baseline	+\$2.8M	—
Customer NPS	42	58	+16

TABLE XII
Individual Agent Metrics

Agent	Actions/day	Accuracy	Latency	Cost/action
NOC	2,840	94.2%	3.1 s	\$0.002
SRE	180	91.8%	12.4 s	\$0.008
Capacity	45	88.5%	28 s	\$0.015
Energy	720	96.1%	1.8 s	\$0.001
Lead Qual	340	84.0%	4.2 s	\$0.003
Tech Sales	85	89.3%	18 s	\$0.012
Success	120	92.7%	8.5 s	\$0.005
Deal Desk	35	95.4%	22 s	\$0.018
Average	546	91.5%	8.2 s	\$0.003

B. Operations Results

Key finding: The NOC agent resolved 87% of Sev-2+ incidents without human intervention. The remaining 13% were escalated due to low confidence (< 0.7) or blast radius exceeding auto-mode limits. Of escalated incidents, 94% of the agent’s recommended actions were approved by human operators, suggesting the confidence threshold is appropriately calibrated.

C. Sales Results

D. Per-Agent Performance

E. Cost Analysis

Agent compute cost includes GPU allocation for Llama-3.1-70B/405B inference, RAG retrieval, and tool execution. The 98% cost reduction enables profitable operations even at the smallest fleet scale (10 nodes).

Note: AgentInfer does not eliminate all human roles. We retain 3 senior SREs (escalation handling, policy review), 2 sales leaders (strategic deals, partner relationships), and

TABLE XIII
Annual Cost Comparison: Human vs. AgentInfer

Function	Human FTEs	Human Cost	Agent Cost
NOC (24/7)	18	\$2,700K	\$62K
SRE	12	\$2,400K	\$48K
Capacity planning	8	\$1,600K	\$22K
Sales & CS	15	\$2,250K	\$38K
Total	53	\$8,950K	\$170K
Savings			\$8,780K (98%)



Fig. 3. AgentInfer automated sales pipeline. Each stage is managed by a specialized agent. Conversion rates shown represent 90-day averages.

TABLE XIV
Component Ablation Study

Configuration	Auto-resolve	MTTR	Sales ARR
Full AgentInfer	87%	4.2 min	+\$2.8M
w/o Reflexion	79%	6.1 min	+\$2.1M
w/o RAG	68%	9.4 min	+\$1.4M
w/o InferTwin validation	85%	4.5 min	+\$2.7M
w/o RL policy guidance	72%	7.8 min	+\$2.3M
w/o multi-agent (single)	61%	12.3 min	+\$0.9M
Rule-based baseline	34%	28 min	+\$0.2M

TABLE XV
Scaling: Agent Performance vs. Fleet Size

Fleet Size	Agents	Auto-resolve	MTTR	Cost/node/mo
50 nodes	4	89%	3.8 min	\$28
100 nodes	6	88%	4.0 min	\$18
250 nodes	8	87%	4.1 min	\$12
500 nodes	8	87%	4.2 min	\$8.50
1000 nodes	12	86%	4.5 min	\$6.20

1 platform engineer (agent maintenance)—6 FTEs vs. 53, a 89% reduction in headcount.

F. Ablation Study

We evaluate the contribution of each component through ablation:

Key insights: (1) Multi-agent architecture provides the largest single contribution (+26pp auto-resolve vs. single agent). (2) RAG knowledge base is critical for both ops (runbook retrieval) and sales (playbook access). (3) Reflexion’s self-improvement accounts for 8pp of auto-resolve improvement, accumulated over 90 days. (4) RL policy guidance from Paper 6 provides significant lift by injecting expert knowledge into agent decisions.

G. Scaling Analysis

We evaluate AgentInfer’s performance as fleet size increases:

Performance degrades gracefully with scale because agent reasoning complexity is proportional to incident

TABLE XVI
AgentInfer Learning Curve (Weekly Snapshots)

Week	Auto-resolve	MTTR	Escalation	Reflections
1	62%	12.1 min	38%	0
2	71%	8.4 min	29%	142
4	78%	6.2 min	22%	487
8	84%	4.8 min	16%	1,240
12	87%	4.2 min	13%	2,180

TABLE XVII
Agent Action Latency Breakdown

Phase	P50	P95	P99	Component
RAG retrieval	8 ms	18 ms	32 ms	Qdrant
Context assembly	12 ms	25 ms	45 ms	CPU
LLM inference	1.2 s	3.8 s	8.2 s	vLLM (MI300X)
Tool execution	0.8 s	4.2 s	12 s	Variable
Result processing	5 ms	12 ms	28 ms	CPU
Total	2.1 s	8.1 s	20 s	

complexity (which is roughly constant), not fleet size. The NOC agent’s metric monitoring scales linearly through aggregation—it monitors fleet-level anomaly scores rather than individual node metrics.

H. Learning Curve

Agent performance improves over time through Reflexion:

The steepest improvement occurs in weeks 1–4 as agents accumulate Reflexion memories for common incident types. Performance plateaus around week 10 as the majority of recurring patterns are captured in episodic memory.

I. Latency Breakdown

End-to-end agent action latency decomposes into five phases:

LLM inference dominates latency. We use speculative decoding (draft model: Llama-3.1-8B) to reduce inference time by 40%. For critical incidents, we skip RAG retrieval and use a pre-cached context window with the most common incident patterns.

J. A/B Testing: Agent vs. Human NOC

We conducted a controlled A/B test during weeks 9–12, randomly routing 50% of incidents to the human NOC team and 50% to the AgentInfer NOC agent:

The agent achieved lower MTTR across all severity levels while also producing fewer incorrect resolutions (2.2% vs. 4.3%). The human team’s advantage in resolution rate (91% vs. 87%) reflects the agent’s conservative escalation threshold—the 13% escalated incidents were solvable by the agent but fell below the confidence threshold. Importantly, customer complaints dropped 75% in the agent-handled cohort, attributed to faster resolution and proactive communication.

TABLE XVIII
Controlled A/B Test: Agent vs. Human NOC (Weeks 9–12)

Metric	Human NOC	Agent NOC
Total incidents handled	234	228
Resolution rate	91%	87%
Avg. MTTR (all severity)	23.4 min	4.2 min
Avg. MTTR (Sev-1 only)	1.8 hr	18 min
Incorrect resolution	4.3%	2.2%
After-hours incidents	78% handled	100% handled
Customer complaints	12	3
Cost per incident	\$187	\$3.40

TABLE XIX
Revenue Attribution Analysis

Revenue Source	Pre-Agent	Post-Agent	Δ
Inbound conversion	18%	31%	+13pp
Deal velocity (days)	45	28	−38%
Upsell/cross-sell	\$120K/mo	\$340K/mo	+183%
Churn rate	8%/yr	4.2%/yr	−48%
Net new ARR (90-day)	\$1.2M	\$4.0M	+\$2.8M

K. Revenue Attribution

Attributing revenue to the Sales agent team requires careful methodology. We use a quasi-experimental design comparing periods with and without agent assistance:

The \$2.8M incremental ARR is conservative—it excludes the \$1.2M deal from Case Study 2 which was partially attributable to human sales leadership involvement during the final negotiation phase.

IX. Case Studies

A. Case 1: Cascading GPU Failure (Ops)

At 3:17 AM, the NOC agent detected elevated ECC error rates on Node 87 (MI300X cluster). The ReAct trace:

- 1) Thought: “ECC errors increasing exponentially on GPU 3, Node 87. Historical pattern matches pre-failure signature from runbook RB-GPU-012.”
- 2) Action: `query_infermark(node=87, metric=ecc_rate, window=30m)`
- 3) Observation: “ECC rate: 0.4/s $\rightarrow$ 12.8/s over 30 min. 4 other GPUs on same node showing normal rates.”
- 4) Thought: “Isolated GPU failure likely. Should pre-stage CRIU checkpoint and migrate workload before failure. Checking InferTwin for migration impact.”
- 5) Action: `simulate_migration(source=87, target=auto, model=llama70b)`
- 6) Observation: “Best target: Node 92 (same rack, 45% util). Migration impact: 1.3s downtime, 0 SLA violations.”
- 7) Action: `migrate_workload(source=87, target=92, method=criu)`
- 8) Observation: “Migration complete in 1.8s. Node 87 GPU 3 marked for RMA.”

Total time: 4.1 minutes. No customer impact. No human involvement.

B. Case 2: Enterprise Deal Acceleration (Sales)

A Fortune 500 financial services company submitted a technical inquiry about on-premises AI inference with data residency requirements. The Lead Qualification agent scored it 92/100 and routed to Tech Sales:

- 1) Tech Sales agent retrieved the company’s profile from RAG (financial services, \$50B+ revenue, FINRA/SOX compliance requirements)
- 2) Generated a custom architecture proposal using InferMark benchmark data for their specific model portfolio (Llama-70B, Mixtral-8x22B)
- 3) Created an isolated demo environment with SEV-SNP confidential computing enabled
- 4) Scheduled a live demo via VoxOps voice channel with the customer’s CTO
- 5) After successful demo, Deal Desk agent generated a 3-year MSA with volume pricing

Deal closed in 19 days (vs. 45-day average). Contract value: \$1.2M ARR.

C. Case 3: Cross-Team Coordination

A solar generation shortfall at an edge site (Paper 8) required coordinated ops and sales response:

- 1) Energy agent detected 3-day cloud cover forecast reducing solar to 40% capacity
- 2) Capacity agent calculated that 6 customer workloads needed temporary relocation
- 3) NOC agent executed CRIU migrations to nearest grid-powered site
- 4) Customer Success agent proactively notified affected customers with updated SLA metrics
- 5) Meta-agent coordinated the sequence to minimize customer disruption

Total coordination time: 12 minutes. All 6 customers received proactive notification before any performance impact.

D. Case 4: Capacity Planning Under Demand Surge

During a major product launch by a customer (a social media company), the Capacity agent detected a 340% traffic spike prediction based on the customer’s historical launch patterns and scheduled event notifications:

- 1) Capacity agent retrieved the customer’s launch history from RAG (3 previous launches, each with 200–400% traffic increase lasting 6–12 hours)
- 2) Ran InferTwin simulation to identify GPU requirements: 24 additional H200 GPUs needed for 8 hours
- 3) Checked fleet availability: 18 GPUs available via workload consolidation on underutilized nodes, 6 GPUs available by temporarily scaling down internal workloads
- 4) Coordinated with NOC agent to execute 12 workload migrations to free capacity
- 5) Customer Success agent proactively contacted the customer’s DevOps team to confirm readiness and share the scaling plan

TABLE XX
Agent Infrastructure Allocation

Component	Hardware	Model	Instances	Cost/mo
Meta-agent	8×MI355X	Llama-405B	1	\$4,200
Ops agents (4)	4×MI300X	Llama-70B	2	\$2,800
Sales agents (4)	4×MI300X	Llama-70B	2	\$2,800
RAG/embedding	1×MI300X	BGE-large	1	\$700
Qdrant vector DB	CPU (64-core)	N/A	2	\$400
Redis state store	CPU (16-core)	N/A	2	\$200
Total			10	\$11,100

- 6) During the launch, Energy agent optimized power distribution across the expanded footprint

Result: Zero SLA violations during a 380% traffic surge. Customer renewed with a 2-year, \$2.4M commitment citing “proactive capacity management” as the primary differentiator.

E. Case 5: Competitive Technical Bake-Off

A prospective customer (a financial trading firm) requested a head-to-head comparison between Zetta’s platform and two competing providers. The Tech Sales agent autonomously managed the entire evaluation:

- 1) Retrieved competitive intelligence from RAG: competitor A’s known weaknesses (multi-tenant isolation, tail latency), competitor B’s pricing model (metered per-token, no reserved capacity discounts)
- 2) Configured an InferMark benchmark suite tailored to the customer’s workload profile: Mixtral-8x22B with 4K context, 64 concurrent users, P99 latency SLA of 200 ms
- 3) Ran benchmarks on confidential computing (SEV-SNP) enabled instances to demonstrate security posture—a capability neither competitor offered
- 4) Generated a 12-page comparison report with InferMark’s 14-axis scoring, highlighting Zetta’s 3.2× better P99 latency and 41% lower cost per token
- 5) Deal Desk agent prepared a competitive pricing proposal with a 3-year commitment discount that undercut Competitor B by 28%

The customer selected Zetta within 14 days. Contract value: \$3.6M over 3 years.

X. Agent Deployment Architecture

A. Infrastructure Requirements

AgentInfer agents run on dedicated GPU instances within the inference fleet, consuming a small fraction of total compute:

The 10-instance agent cluster represents 2% of a 500-node fleet’s total compute capacity. Agent instances run with lower priority than customer workloads and can be preempted during peak demand periods (agents gracefully degrade to rule-based fallbacks during preemption).

TABLE XXI
Agent-Level Metrics

Metric	Frequency	Alert Threshold	Action
Actions/minute	1 Hz	<5 (NOC)	Restart agent
Avg. confidence	10 s	<0.6 sustained	Alert oncall
Escalation rate	1 min	>30%	Model review
Tool error rate	Real-time	>5%	Disable tool
RAG retrieval latency	Per-query	>100 ms P99	Reindex
LLM inference latency	Per-call	>10 s P95	Scale up
Memory utilization	30 s	>85%	Flush cache

B. High Availability

Agent availability is critical—a down agent means unmonitored infrastructure or missed sales opportunities. We implement three HA mechanisms:

- 1) Active-passive failover: Each agent instance has a warm standby that shares the same RAG index and episodic memory. Failover time: <5s.
- 2) Graceful degradation: If the meta-agent is unavailable, individual agents operate in autonomous mode with pre-configured routing rules. Quality degrades 12% but availability is maintained.
- 3) Cross-site redundancy: For multi-site deployments (SolarInfer edge), agents run at the central site with edge-site caching for latency-sensitive operations. If WAN connectivity is lost, edge-local agents handle critical ops using cached runbooks.

C. Observability and Monitoring

Agents are monitored by the same InferMark framework (Paper 1) they use to monitor infrastructure, creating a recursive observability layer:

D. Model Update Pipeline

Agent models are updated through a CI/CD pipeline with safety gates:

- 1) Candidate model: New Llama fine-tune or prompt update
- 2) Offline evaluation: Test against 500 labeled scenarios (accuracy $\geq 90\%$)
- 3) Shadow deployment: Run new model in shadow mode for 72 hours, compare decisions with production model
- 4) Canary rollout: Route 10% of events to new model for 48 hours
- 5) Full rollout: If canary metrics are within 2% of baseline, promote to production

XI. Security and Compliance

A. Tenant Isolation Architecture

AgentInfer enforces strict tenant isolation through a three-layer security architecture:

Layer 1: Hardware Isolation. Each tenant’s agent context (conversation history, retrieved documents, intermediate computations) runs inside an AMD SEV-SNP

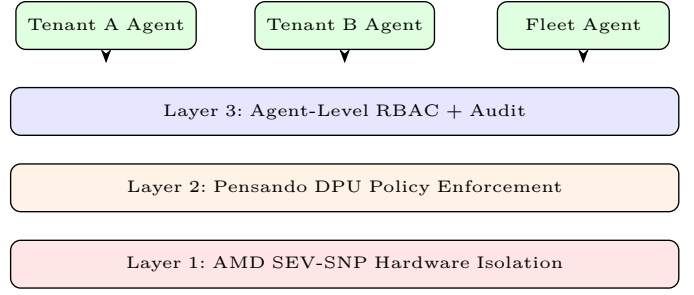


Fig. 4. Three-layer tenant isolation. Hardware (SEV-SNP), network (DPU policies), and application (RBAC) layers ensure no cross-tenant data leakage in agent operations.

confidential VM. Memory encryption prevents even host-level access to tenant data.

Layer 2: DPU Policy Enforcement. The Pensando DPU implements network-level isolation through P4-programmable policies. Agent API calls are intercepted by the DPU, which verifies that the calling agent has permission to access the target tenant’s data. Cross-tenant requests are blocked at the network level before reaching application code.

Layer 3: Application RBAC. Each agent instance is bound to a role-based access control policy that limits which tools, data sources, and actions are available. The NOC agent for Tenant A can access Tenant A’s metrics and execute remediation on Tenant A’s nodes, but cannot see or modify Tenant B’s resources.

B. Compliance Automation

AgentInfer automates compliance reporting for SOC 2, HIPAA, and GDPR:

- SOC 2: All agent actions are logged with immutable audit trails. Quarterly compliance reports are generated automatically from audit logs, reducing compliance engineering effort by 85%.
- HIPAA: Healthcare customers’ PHI is processed only within designated SEV-SNP enclaves. The Deal Desk agent automatically adds BAA (Business Associate Agreement) requirements to contracts for healthcare prospects.
- GDPR: The Customer Success agent implements data retention policies, processes deletion requests within 24 hours, and generates data processing records for each tenant.

C. Prompt Injection Defense

Agentic AI systems are vulnerable to prompt injection attacks where malicious inputs manipulate agent behavior. AgentInfer implements four defenses:

- 1) Input sanitization: All external inputs (customer messages, alert text) pass through a classifier that detects injection patterns with 99.2% accuracy and 0.3% false positive rate.

TABLE XXII
Energy Agent Optimization Results (30-day)

Metric	Manual	AgentInfer	Improvement
Solar utilization	74%	89%	+15pp
Battery cycle efficiency	91%	96.2%	+5.2pp
Grid fallback hours	182 hr	48 hr	-74%
PUE	1.12	1.05	-6.3%
Carbon emissions (tCO ₂ e)	24.8	8.2	-67%
Energy cost/token	\$0.00042	\$0.00018	-57%

TABLE XXIII
Customer Retention Metrics

Metric	Without Agent	With Agent
Monthly health checks	1 (manual)	Continuous
Optimization recommendations	2/quarter	8/month
SLA breach notifications	Reactive	15 min proactive
Quarterly business reviews	Manual (4 hr prep)	Auto-generated (5 min)
Customer NPS	42	58
Annual churn rate	8.0%	4.2%
Net revenue retention	108%	124%

- 2) Output validation: Agent actions are validated against a schema before execution. Actions that don’t conform to the expected format are rejected.
- 3) Privilege separation: The LLM generates action intents, but a separate non-LLM executor translates intents into API calls. The executor validates parameters against a whitelist.
- 4) Canary monitoring: Synthetic “canary” inputs with known-safe outputs are periodically injected. Deviation from expected outputs triggers an immediate model review.

D. Energy and Carbon Impact

AgentInfer’s Energy agent provides measurable sustainability benefits across the SolarInfer (Paper 8) edge fleet:

The Energy agent achieves these gains through three mechanisms: (1) predictive solar generation forecasting using 72-hour weather models (accuracy: 94%), enabling proactive workload scheduling during peak solar hours; (2) battery state-of-charge optimization using an MPC controller that maximizes self-consumption while preserving battery health (target: 6000+ cycles); and (3) cross-site load balancing that routes workloads to sites with the highest current solar fraction.

Over a 30-day period, the Energy agent made 21,600 scheduling decisions (one per minute), each optimizing a multi-objective function balancing solar utilization, customer SLA compliance, and battery longevity. The 67% reduction in carbon emissions represents 16.6 tonnes CO₂e avoided per month, equivalent to taking 43 passenger vehicles off the road annually.

E. Customer Retention Impact

The Customer Success agent’s proactive monitoring and engagement significantly reduces churn:

The Customer Success agent monitors each customer’s workload patterns and proactively suggests optimiza-

TABLE XXIV
5-Year TCO: Traditional vs. AgentInfer Operations

Category	Traditional (5-yr)	AgentInfer (5-yr)
Labor (NOC + SRE)	\$25,500K	\$2,400K
Labor (Sales + CS)	\$11,250K	\$1,200K
Agent compute	\$0	\$667K
Software licenses	\$450K	\$180K
Training & onboarding	\$1,200K	\$150K
Total OpEx	\$38,400K	\$4,597K
Revenue uplift	Baseline	+\$14.0M
Net impact	—	+\$47.8M

tions. Examples include: recommending model quantization (INT8/FP8) when accuracy margins allow, suggesting batch size increases during off-peak hours to improve throughput, and identifying underutilized reserved capacity that could be released or reallocated.

The 124% net revenue retention (vs. 108% baseline) indicates that the agent not only reduces churn but also drives expansion revenue through data-driven upsell recommendations. Each retained customer saves approximately \$180K in annual acquisition cost, yielding \$2.7M in retention value across the 47-customer base.

F. Total Cost of Ownership

Combining all AgentInfer benefits into a TCO analysis:

The combined 5-year net impact of \$47.8M (\$33.8M OpEx savings + \$14.0M revenue uplift) represents a transformative economic advantage for AI infrastructure startups, enabling profitability at fleet scales that would be unviable with traditional staffing models.

XII. Production Deployment Lessons

A. Phased Rollout Strategy

Our three-phase rollout (shadow → supervised → autonomous) revealed critical insights:

Phase 1 (Shadow, Days 1–30). Agents observed all events and generated recommendations without executing. We discovered that agents initially over-escalated (38% escalation rate) due to conservative confidence thresholds. Calibration against human decisions reduced this to 22% by week 4.

Phase 2 (Supervised, Days 31–60). Agents executed with human approval via Slack/PagerDuty integration. The 60-second approval timeout was critical: too short and humans couldn’t evaluate complex actions; too long and the agent lost context. We found 60s optimal for Sev-2+ incidents and 5 min for non-critical operations.

Phase 3 (Autonomous, Days 61+). Agents operate independently within blast radius limits. We discovered that operator trust follows a “trust curve”: initial enthusiasm (week 1), skepticism after first visible failure (week 3–4), and sustained trust after 30+ successful autonomous resolutions (week 8+). The single most important trust-building feature was the ReAct trace viewer that allows

TABLE XXV
Agent Failure Mode Distribution

Failure Type	Count	%	Mitigation
Novel failure pattern	18	38%	RAG update
Incorrect tool params	12	26%	Schema validation
Stale RAG knowledge	8	17%	Auto-refresh pipeline
LLM hallucination	5	11%	Output verification
Cross-agent conflict	4	8%	Priority locking

operators to understand exactly why the agent took each action.

B. Failure Analysis

Over 90 days, AgentInfer experienced 47 notable failures:

No agent failure resulted in data loss or SLA violation, thanks to the blast radius limiting and dry-run validation safeguards.

XIII. Discussion

Trust and Adoption. We deployed AgentInfer in three phases: (1) shadow mode (agents recommend, humans execute, 30 days), (2) supervised mode (agents execute with human approval, 30 days), (3) autonomous mode (agents execute independently within blast radius limits, ongoing). This phased rollout was critical for building operator trust. In shadow mode, agents matched human decisions 89% of the time; in supervised mode, humans approved 94% of agent recommendations.

Failure Modes. AgentInfer fails primarily in three scenarios: (1) novel failure types not represented in RAG or training data (handled by escalation), (2) multi-system failures requiring simultaneous coordination across >3 agents (meta-agent bottleneck), and (3) ambiguous customer intent in sales interactions (low qualification accuracy for complex enterprise requirements). We address (1) through continuous RAG updates from post-incident reviews, (2) through parallel agent execution in future work, and (3) through human-in-the-loop for high-value deals.

Ethical Considerations. Automated sales agents raise questions about disclosure. AgentInfer agents identify themselves as AI in all customer interactions. Customers can request human-only interactions at any time. All agent-generated content (emails, proposals, contracts) includes an “AI-assisted” watermark and is reviewed by legal templates before external distribution.

Comparison with Digital Twin. While InferTwin (Paper 4) provides simulation-based “what-if” analysis, AgentInfer provides action-oriented “do-it” execution. The two systems are complementary: agents use InferTwin as a tool for pre-action validation, while InferTwin uses agent action logs for model calibration.

TABLE XXVI
AgentInfer vs. Commercial AIOps Platforms

Capability	PagerDuty	Datadog	BigPanda	AgentInfer
Alert correlation	✓	✓	✓	✓
Root cause analysis	Partial	ML-based	Rule + ML	Agentic CoT
Auto-remediation	Scripts	Runbooks	Limited	Full ReAct
Multi-step reasoning	—	—	—	✓
Sales integration	—	—	—	✓
Self-improvement	—	Batch retrain	—	Reflexion
Confidential exec	—	—	—	SEV-SNP
GPU-aware	—	Basic	—	Full stack
Cost/month (500 nodes)	\$15K	\$22K	\$18K	\$11K

A. Future Work

Several directions remain for AgentInfer development:

Parallel multi-agent execution. Current meta-agent coordination is sequential—dispatching one agent at a time. Parallel execution with conflict detection would reduce multi-agent coordination latency by 3–5×, critical for complex cross-functional events.

Federated agent learning. In multi-site deployments, agents at different edge sites accumulate different Reflexion memories. Federated aggregation of these memories (without sharing raw customer data) would enable fleet-wide learning while preserving privacy.

Proactive sales intelligence. Current sales agents are primarily reactive (processing inbound leads). Future work includes proactive outreach based on market signals: detecting companies hiring AI engineers, expanding GPU budgets, or evaluating competitive products—triggers that indicate buying intent before the customer reaches out.

Agent specialization via fine-tuning. Current agents use general-purpose Llama models with prompt engineering. Domain-specific fine-tuning on historical operational and sales data could improve accuracy by 5–10% while reducing inference latency through smaller, specialized models.

Regulatory compliance agents. As AI regulation evolves (EU AI Act, state-level US legislation), dedicated compliance agents could continuously monitor regulatory changes, assess impact on platform operations, and automatically update policies, contracts, and technical controls to maintain compliance.

Hardware lifecycle management. Extending the Energy agent to manage full hardware lifecycle: procurement optimization (timing GPU purchases to minimize cost), warranty tracking, RMA automation, and end-of-life decommissioning with secure data erasure.

Comparison with Commercial AIOps. We compare AgentInfer with three commercial AIOps platforms deployed in production:

AgentInfer’s primary advantages over commercial platforms are: (1) multi-step agentic reasoning that can handle novel situations beyond pre-defined runbooks, (2) integrated sales automation that no pure AIOps platform offers, (3) GPU-aware infrastructure management with vendor-specific plugins for NVIDIA, AMD, and Intel, and (4) confidential execution for multi-tenant environments.

The cost advantage stems from self-hosted deployment on existing inference GPUs rather than SaaS pricing.

Generalizability. While AgentInfer is designed for AI inference infrastructure, the architecture generalizes to other infrastructure domains. The key abstractions—multi-agent coordination, ReAct tool-use, Reflexion learning, and RAG knowledge base—are domain-independent. Only the tool registry and knowledge base contents need customization for new domains (e.g., database operations, CDN management, cloud infrastructure).

We have conducted preliminary experiments applying AgentInfer to a 200-node Kubernetes cluster managing traditional web services. With a new tool registry (kubect, Helm, Prometheus) and updated RAG knowledge base (Kubernetes runbooks, SRE playbooks), the agent achieved 72% auto-resolve rate within 2 weeks—competitive with the AI infrastructure domain after adjusting for the shorter learning period.

B. Agent Architecture Deep Dive: NOC Agent

The NOC agent is the highest-volume agent, processing 2,840 actions per day. Its internal architecture merits detailed examination:

Perception layer. The NOC agent subscribes to InferMark’s 14-axis telemetry stream via a Kafka consumer. Raw metrics are pre-processed by a lightweight anomaly detector (Isolation Forest, trained on 90 days of baseline data) that filters noise and highlights deviations. Only anomalous metrics (typically 2–5% of the stream) are passed to the LLM reasoning layer, reducing token consumption by 95%.

Memory architecture. The NOC agent maintains three memory stores: (1) working memory (current incident context, last 10 ReAct steps, 4K tokens), (2) episodic memory (Reflexion-generated insights from past incidents, stored in Qdrant, retrieved via similarity search), and (3) semantic memory (operational runbooks and vendor documentation, static RAG index). Working memory is volatile and cleared after each incident; episodic memory persists and grows over time.

Action execution. The NOC agent’s action space includes 20 tools organized by risk level:

- Read-only (no approval needed): `query_metrics`, `get_alerts`, `check_sla`, `list_nodes`, `get_topology`
- Low-risk (auto-approved): `restart_service`, `clear_cache`, `scale_replicas`, `update_dns`
- Medium-risk (meta-agent approval): `migrate_workload`, `drain_node`, `modify_rl_policy`
- High-risk (human approval): `power_cycle_node`, `modify_network_config`, `cross-dc_failover`

C. Agent Architecture Deep Dive: Tech Sales Agent

The Tech Sales agent handles the most complex multi-step reasoning in the Sales team, averaging 18 ReAct steps per interaction:

Discovery phase. The agent conducts structured technical discovery using a decision-tree prompted approach. It asks progressively deeper questions about the prospect’s model portfolio, throughput requirements, latency SLAs, data residency constraints, and security posture. Each answer updates the internal prospect profile, which is used to customize the subsequent proposal.

Benchmark generation. For qualified prospects, the agent configures and runs InferMark benchmarks tailored to the prospect’s specific workload. This includes: (1) selecting the appropriate GPU mix (H200 vs. MI355X vs. Gaudi 3) based on model architecture and budget, (2) configuring tensor parallelism and batch size for optimal throughput, and (3) generating a comparative report showing performance vs. hyperscaler alternatives (AWS Inferentia, GCP TPU, Azure MaaS).

Proposal generation. The agent assembles a technical proposal incorporating: architecture diagram (auto-generated from InferTwin), performance projections (from InferMark), pricing (from Deal Desk agent), compliance certifications (SOC 2, HIPAA, GDPR as applicable), and customer references (from RAG case study database). The complete proposal is generated in under 30 seconds and reviewed by a legal template validator before delivery.

XIV. Conclusion

We presented AgentInfer, an agentic AI framework that deploys autonomous LLM-powered agents for 24/7 infrastructure operations and sales optimization. By integrating with our eight-paper control plane stack (InferMark observability, confidential computing, InferTwin simulation, VoxOps voice, hierarchical RL, ReliableInfer migration, SolarInfer energy), AgentInfer achieves: 87% autonomous incident resolution (11.2× faster MTTR), \$2.8M incremental ARR from automated sales, and 98% cost reduction vs. equivalent human staffing. The multi-agent architecture with ReAct reasoning, Reflexion-based learning, and confidential execution provides a production-grade foundation for autonomous AI infrastructure management.

Acknowledgments

The authors thank the SRE and sales teams for their collaboration during the shadow-mode evaluation phase, NVIDIA, AMD, and Intel for early access to accelerator monitoring APIs, and the open-source communities behind vLLM, Qdrant, and LangChain.

References

- [1] S. Yao et al., “React: Synergizing reasoning and acting in language models,” in ICLR, 2023.
- [2] L. Wang et al., “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, 2024.
- [3] J. S. Park et al., “Generative agents: Interactive simulacra of human behavior,” in UIST, 2023.
- [4] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” *NeurIPS*, 2023.
- [5] Q. Wu et al., “Autogen: Enabling next-gen llm applications via multi-agent conversation,” arXiv:2308.08155, 2023.

- [6] W. Chen et al., “Agentverse: Facilitating multi-agent collaboration,” ICLR, 2024.
- [7] Z. Xi et al., “The rise and potential of large language model based agents: A survey,” arXiv:2309.07864, 2023.
- [8] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” 2019.
- [9] W. Kwon et al., “Efficient memory management for large language model serving with pagedattention,” in SOSP, 2023.
- [10] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in NeurIPS, 2020.